

Algoritmi-Modulo algoritmi per bioinformatica

Algoritmi elementari per grafi-Capitolo 20 CLRS

Roberto Posenato

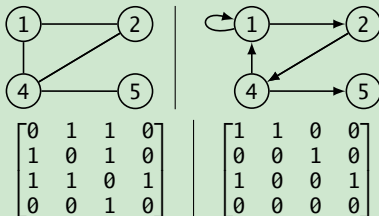
versione 2026-1, 23/05/2026

- 1 Visita in ampiezza
- 2 Visita in profondità
- 3 Ordinamento Topologico
- 4 Componenti Fortemente Connesse
- 5 Esercizi

- Si assume che i concetti matematici inerenti ai grafi siano già noti dal corso ‘Metodi informativi’.
- Qui si ricorda solo quali sono le due tecniche principali per rappresentare i grafi.
- Ci sono due modi per rappresentare la struttura dei grafi (non) orientati: **matrice di adiacenza** o **liste di adiacenza**.

- Dato un grafo di n nodi, la **matrice di adiacenza** è una matrice $n \times n$ dove
 - sia le righe sia le colonne sono etichettate con i nomi dei nodi.
 - la cella di riga i e colonna j , ovvero la cella di coordinate (i, j) indica se esiste l'arco dal nodo i al nodo j : 1 o 0.
- Nel caso di grafo non orientato, la matrice è simmetrica.

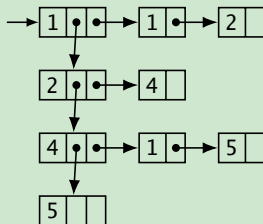
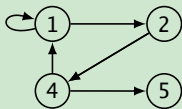
Esempio 1



- La memoria richiesta per rappresentare un grafo è $\Theta(n^2)$.
- Verificare se un arco esiste richiede tempo $\Theta(1)$.
- Accedere a tutti gli adiacenti di un nodo richiede tempo $\Theta(n)$.
- Identificare tutti gli archi richiede tempo $\Theta(n^2)$.

- Dato un grafo di n nodi, la rappresentazione con **liste di adiacenza** prevede di avere una lista che rappresenta i nodi e, per ciascun elemento della lista, una lista collegata che rappresenta i suoi nodi adiacenti.

Esempio 2



- Se il numero di nodi è fisso, la lista principale può essere un array Adj di riferimenti a liste di nodi.
 - $Adj[v]$: lista di adiacenza di v . Contiene tutti i vertici u tali che $(v, u) \in E$.
- La memoria richiesta per rappresentare un grafo è $\Theta(n + m)$.
- Verificare se un arco (u, v) esiste richiede tempo $\Theta(d(u))$ (nel caso peggiore $\Theta(n)$).
- Accedere a tutti gli adiacenti di un nodo u richiede tempo $\Theta(d(u))$ (nel caso peggiore $\Theta(n)$).
- Identificare tutti gli archi richiede tempo $\Theta(n + m)$.
- Nel caso di grafi non orientati ci sono due elementi per ogni arco (u, v) , uno nella lista di u e l'altro nella lista di v .

Nota!

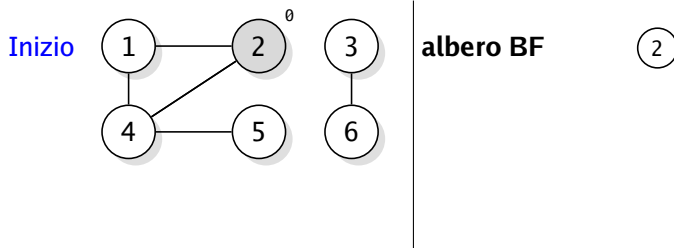
Accettando d'impiegare più memoria del necessario, è possibile implementare le liste di adiacenza in modo da diminuire i tempi di esecuzione per verificare se un nodo/arco esiste a un fattore quasi costante. In cosa si basa tale implementazione?

- La **visita in ampiezza** (*breadth-first search*) è un algoritmo di ricerca che fornisce uno schema di base per alcuni algoritmi importanti sui grafi:
 - Algoritmo di Prim per l'albero di connessione minimo.
 - Algoritmo di Dijkstra per i cammini minimi da sorgente unica.
 - ...
- Dato un grafo $G = (V, E)$ e un nodo $s \in V$ (detto **sorgente**), la *visita in ampiezza*:
 - ispeziona gli archi di G per *scoprire* tutti i nodi raggiungibili da s
 - L'ispezione è fatta in ordine: per $k = 1, 2, \dots, n - 1$, *scopre* i nodi che stanno a distanza k a partire dai nodi che stanno a distanza $k - 1$.
 - calcola la distanza minima da s a ciascun nodo raggiungibile.
 - genera un albero, detto **BF** (*breadth-first tree*) con radice s e tutti i nodi raggiungibili da s .

Grafi

Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

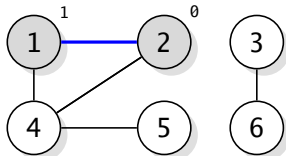


Grafi

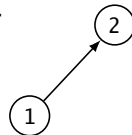
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

1 passo



albero BF

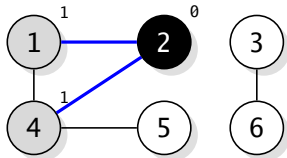


Grafi

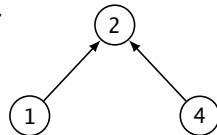
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

2 passo



albero BF

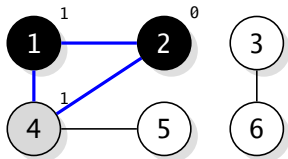


Grafi

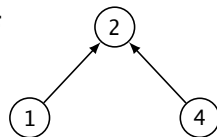
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

3 passo



albero BF

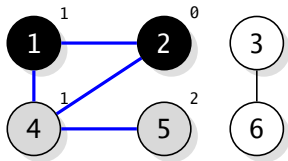


Grafi

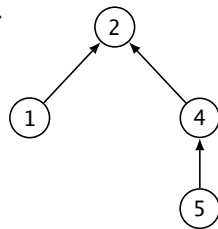
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

4 passo



albero BF

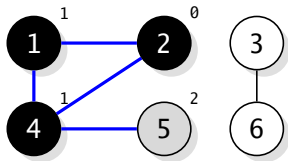


Grafi

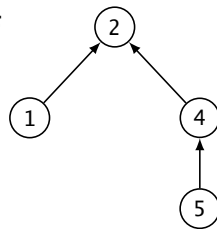
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

5 passo



albero BF

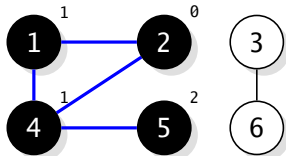


Grafi

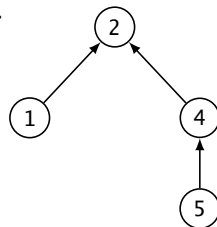
Visita in ampiezza da una singola sorgente

- I nodi sono bianchi.
- Un nodo diventa grigio quando viene scoperto.
- Un nodo diventa nero quando tutti i suoi vicini sono stati scoperti (sono grigi o neri).
- Nell'esempio seguente, il nodo 2 è il nodo sorgente.

6 passo



albero BF



Grafi

Algoritmo visita in ampiezza (BFS)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

1: **foreach** $u \in V \setminus \{s\}$ **do**

2: $u.color := \text{WHITE}$

3: $u.d := \infty$

4: $u.\pi := \text{NULL}$

// d è la distanza da s

// π è il nodo precedente nel cammino da s a u

5: $s.color := \text{GRAY}$

6: $s.d := 0$

7: $s.\pi := \text{NULL}$

8: $Q := \emptyset$

// è una coda con schema FIFO che contiene i nodi grigi

9: $Q.\text{ENQUEUE}(s)$

10: **while** $Q \neq \emptyset$ **do**

11: $u := Q.\text{DEQUEUE}()$

12: **foreach** $v \in G.\text{Adj}[u]$ **do**

13: **if** $v.color == \text{WHITE}$ **then**

14: $v.color := \text{GRAY}$

15: $v.d := u.d + 1$

16: $v.\pi := u$

17: $Q.\text{ENQUEUE}(v)$

18: $u.color := \text{BLACK}$

Grafi

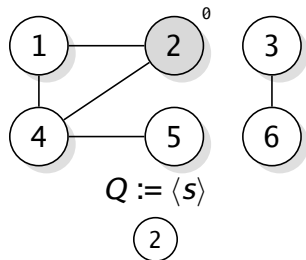
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo esecuzione linee 1-9



Grafi

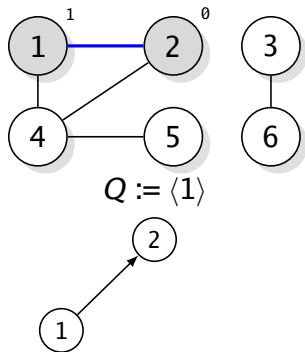
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo I ciclo foreach riga 12, $u == 2$



Grafi

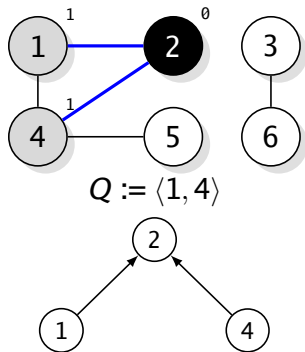
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo II ciclo foreach riga 12, $u == 2$



Grafi

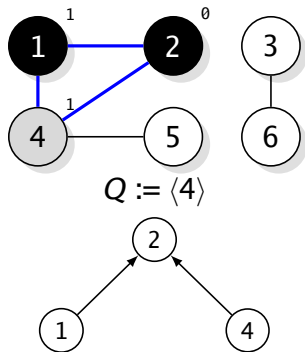
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo 2 cicli foreach riga 12, $u == 1$



Grafi

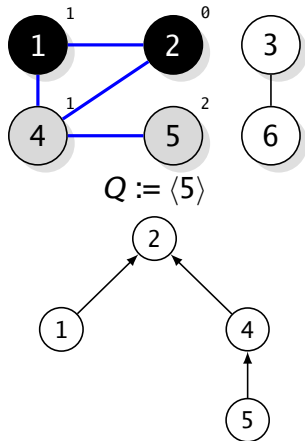
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo I ciclo foreach riga 12, $u == 4$



Grafi

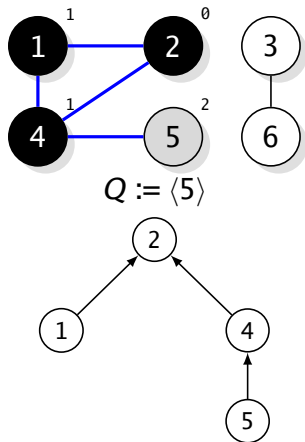
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo 3 cicli foreach riga 12, $u == 4$



Grafi

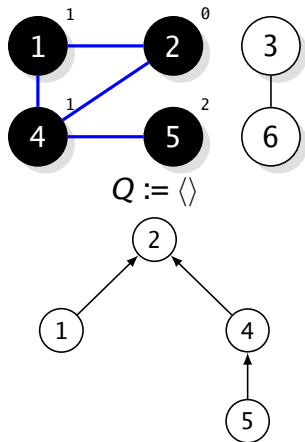
Visita in ampiezza: BFS($G, 2$)

Algoritmo: BFS(G, s)

Input: $G = (V, E)$, $s \in V$

```
1: foreach  $u \in V \setminus \{s\}$  do
2:    $u.color := WHITE$ 
3:    $u.d := \infty$ 
4:    $u.\pi := NULL$ 
5:  $s.color := GRAY$ 
6:  $s.d := 0$ 
7:  $s.\pi := NULL$ 
8:  $Q := \emptyset$ 
9:  $Q.ENQUEUE(s)$ 
10: while  $Q \neq \emptyset$  do
11:    $u := Q.DEQUEUE()$ 
12:   foreach  $v \in G.Adj[u]$  do
13:     if  $v.color == WHITE$  then
14:        $v.color := GRAY$ 
15:        $v.d := u.d + 1$ 
16:        $v.\pi := u$ 
17:        $Q.ENQUEUE(v)$ 
18:    $u.color := BLACK$ 
```

Dopo I ciclo foreach riga 12, $u == 5$



- Il primo ciclo **for** (linee 1–4) e linee 5–7 preparano ciascun nodo. Sono $O(|V|)$ operazioni elementari in totale (O e non Θ : l'inizializzazione può essere effettuata in modo pigro, in $O(1)$).
- Il ciclo **while** (linee 10–18) si ripete finché ci sono nodi grigi.
- L'invariante di ciclo è quindi *Quando viene eseguito il test ciclo **while**, la coda Q contiene solo nodi grigi.* (dimostrazione lasciata per esercizio)
- Ogni nodo diventa grigio solo una volta.
- Quindi la coda ha lunghezza massima $|V|$. # cicli **while** $\leq |V|$.
- Operazioni di enqueue e dequeue si possono realizzare in $O(1)$.
- Per ciascun nodo u , il ciclo **foreach** esamina tutti i suoi adiacenti.
- Ogni lista di adiacenza viene esaminata solo una volta.
- In un grafo non orientato, ogni arco viene quindi considerato 2 volte.
- Nel suo complesso, **while** costa $O(|V| + |E|)$.
- La complessità finale è quindi $O(|V| + |E|)$.

Definizione 1 (Grafo pesato)

Un grafo $G = (V, E)$ si dice **pesato** quando esiste una funzione **peso**:

$$\omega: E \rightarrow \mathbf{R}$$

che associa a ogni arco un valore detto **peso** o **valore** dell'arco.

Un grafo non pesato può essere visto come grafo pesato dove la funzione peso è

$$\omega(e) = 1, \quad \forall e \in E$$

Definizione 2 (Distanza di cammino minimo e cammino minimo)

Si definisce **distanza di cammino minimo** $\delta(s, v)$ da s a v come la minima somma dei pesi degli archi in un cammino qualsiasi $s \overset{P}{\rightsquigarrow} v$:

$$\delta(s, v) = \min_{P | s \overset{P}{\rightsquigarrow} v} \sum_{e \in P} \omega(e)$$

Se non c'è un cammino, allora $\delta(s, v) = \infty$.

Un cammino $s \overset{P}{\rightsquigarrow} v$ di lunghezza $\delta(s, v)$ è detto **cammino minimo** da s a v .

Lemma 1

Se $G = (V, E)$ è un grafo **non pesato** e $s \in V$, allora per qualsiasi arco $(u, v) \in E$ vale

$$\delta(s, v) \leq \delta(s, u) + 1$$

Dimostrazione.

Se u è raggiungibile da s , lo è anche v .

Sia $s \overset{P'}{\rightsquigarrow} u$ il cammino minimo da s a u .

Il cammino minimo $s \overset{P}{\rightsquigarrow} v$ **non può essere più lungo** del cammino minimo $s \overset{P'}{\rightsquigarrow} u \rightarrow v$.

Se u non è raggiungibile, allora $\delta(s, u) = \infty$ e la disuguaglianza è ancora vera. □

Lemma 2

*Sia $G = (V, E)$ un grafo **non pesato**, $s \in V$.*

Si assuma che venga eseguito $\text{BFS}(G, s)$.

Al termine di BFS, $v.d \geq \delta(s, v) \forall v \in V$.

Dimostrazione.

Per induzione sul numero di operazioni $\text{ENQUEUE}()$.

Caso base è quando solo il nodo s è in Q : $s.d = 0 = \delta(s, s)$ mentre $v.d = \infty \geq \delta(s, v)$ per tutti gli altri.

Passo induttivo.

Un nodo bianco v viene scoperto durante la visita di u .

Per ipotesi, $u.d \geq \delta(s, u)$.

L'algoritmo fissa $v.d := u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$.

Il valore $v.d$ non verrà più cambiato.



Lemma 3

Si assuma che durante BFS, la coda Q contenga $\langle v_1, v_2, \dots, v_r \rangle$. Allora, $v_r.d \leq v_1.d + 1$ e $v_i.d \leq v_{i+1}.d$ per $i = 1, 2, \dots, r - 1$.

Dimostrazione.

Per induzione sulle operazioni possibili su Q .

Base: $Q = \langle s \rangle$. La tesi è banale.

Passo induttivo.

Se si estrae v_1 , la coda diventa $\langle v_2, \dots, v_r \rangle$. Per ipotesi induttiva valeva

$$v_r.d \leq v_1.d + 1 \quad \text{e} \quad v_1.d \leq v_2.d.$$

Quindi $v_r.d \leq v_2.d + 1$, e l'ordine non decrescente delle distanze nella coda resta valido.

Se durante l'esame di $u = v_1$ si inserisce un nuovo nodo v in coda, allora $v.d = u.d + 1 = v_1.d + 1$.

Dopo l'estrazione di u , il nuovo primo elemento della coda ha distanza almeno $v_1.d$, quindi il nuovo ultimo elemento soddisfa ancora $v.d \leq (\text{nuovo primo elemento}).d + 1$. □

Corollario 1

Se v_i e v_j sono inseriti, in ordine, in coda durante l'esecuzione di BFS, allora $v_i.d \leq v_j.d$.

Teorema 1

BFS scopre tutti i nodi $v \in V$ che sono raggiungibili da s e, alla fine, $v.d = \delta(s, v)$ per ogni $v \in V$.

Inoltre, per qualsiasi nodo v raggiungibile da s , uno dei cammini minimi da s a v è un cammino minimo da s a $v.\pi$ seguito dall'arco $(v.\pi, v)$.

Dimostrazione.

Si dimostra per assurdo. Si assume che la distanza calcolata non è la minima e si determina una contraddizione.

Fra tutti i vertici raggiungibili per cui $v.d \neq \delta(s, v)$, sia v uno con distanza minima $\delta(s, v)$. Per il Lemma 1, $v.d > \delta(s, v)$. v deve essere diverso da s ma raggiungibile.

Sia u il vertice che precede immediatamente v in un cammino minimo $s \overset{P}{\rightsquigarrow} v$. Vale $\delta(s, v) = \delta(s, u) + 1$.

Visto che u precede v in P , $\delta(s, u) < \delta(s, v)$.

Per la scelta di v , si ha quindi $u.d = \delta(s, u)$.

Deve valere quindi $v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1$.

Si consideri ora quando l'algoritmo considera il nodo v come adiacente di u .

Se v è bianco, allora $v.d = u.d + 1$ che contraddice quanto appena stabilito.

Se v è nero, allora v è già stato rimosso dalla coda, quindi $v.d \leq u.d < u.d + 1 < v.d$ per il Corollario 1: contraddizione.

Se v è grigio, è stato colorato grigio durante la visita di un nodo w , rimosso dalla coda prima di u . Quindi $v.d = w.d + 1$.

Per il Corollario 1, $w.d \leq u.d$. Quindi $v.d = w.d + 1 \leq u.d + 1$, ma avevamo $v.d > u.d + 1$: contraddizione.

L'algoritmo pone $v.\pi = u$ e $v.d = u.d + 1$.

Quindi un cammino minimo $s \overset{P}{\rightsquigarrow} v$ si ottiene ricorsivamente considerando un cammino minimo $s \overset{P'}{\rightsquigarrow} v.\pi$ e aggiungendo poi l'arco $(v.\pi, v)$. □

- La **visita in profondità** (*depth-first search*) è un algoritmo che visita il grafo sempre più *in profondità* se possibile.
- Dato un grafo $G = (V, E)$, la *visita in profondità*:
 - ispeziona gli archi di G a partire dall'**ultimo** nodo scoperto v che ha ancora archi non ispezionati;
 - quando tutti gli archi di v sono ispezionati, l'algoritmo ritorna al nodo dal quale v è stato scoperto per ispezionare gli altri archi;
 - Questo processo si ripete finché ci sono nodi da ispezionare (è diverso da visita in ampiezza).

- Come per visita in ampiezza, per ogni nodo si registra il predecessore. Possono generarsi più alberi: **foresta DF** (*depth-first forest*).
- La visita in profondità calcola anche, per ciascun nodo v , gli **istanti**
 - $v.d$ = istante in cui v è scoperto (diventa grigio)
 - $v.f$ = istante in cui v è stato completato (diventa nero).
- L'algoritmo ricorsivo che si propone usa una variabile globale, `time`, per gestire il *tempo* di scoperta/completamento dei nodi. `time` viene incrementata ogni volta che si deve cambiare il colore del nodo corrente.

DFS versione [CLRS]

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ 
3:    $u.\pi := NULL$ 
4:  $time := 0$  // Variabile globale
5: foreach  $u \in V$  do
6:   if  $u.color == WHITE$  then
7:     DFS-VISIT( $G, u$ )
```

Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ 
2:  $u.d := time$ 
3:  $u.color := GRAY$ 
4: foreach  $v \in G.Adj[u]$  do
5:   if  $v.color == WHITE$  then
6:      $v.\pi := u$ 
7:     DFS-VISIT( $G, v$ )
8:  $time := time + 1$ 
9:  $u.f := time$ 
10:  $u.color := BLACK$ 
```

Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE$ ;  $u.\pi := NULL$   
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

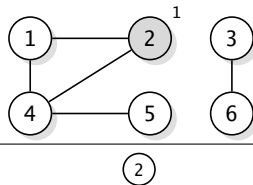
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
7:  $time := time + 1$ ;  $u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(2)

$time := 1$
Stack chiamate DFS-V: $\langle 2 \rangle$



Foresta DF:

Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:     DFS-VISIT( $G, u$ )
```

Algoritmo: DFS-VISIT(G, u)

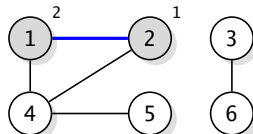
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:     DFS-VISIT( $G, v$ )
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(1)

$time := 2$

Stack chiamate DFS-V: $\langle 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:      $DFS-VISIT(G, u)$ 
```

Algoritmo: DFS-VISIT(G, u)

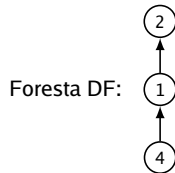
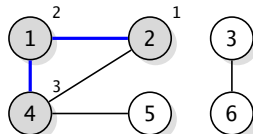
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:      $DFS-VISIT(G, v)$ 
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(u)

$time := 3$

Stack chiamate DFS-V: $\langle 4, 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE$ ;  $u.\pi := NULL$   
  
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

Algoritmo: DFS-VISIT(G, u)

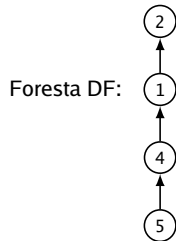
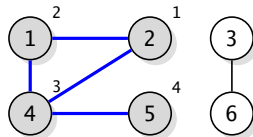
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
  
7:  $time := time + 1$ ;  $u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(5)

$time := 4$

Stack chiamate DFS-V: $\langle 5, 4, 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE$ ;  $u.\pi := NULL$   
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

Algoritmo: DFS-VISIT(G, u)

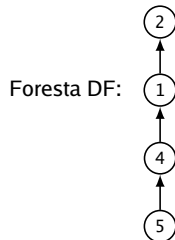
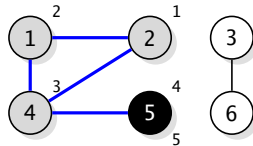
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
7:  $time := time + 1$ ;  $u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(5)

$time := 5$

Stack chiamate DFS-V: $\langle 5, 4, 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:     DFS-VISIT( $G, u$ )
```

Algoritmo: DFS-VISIT(G, u)

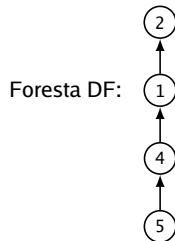
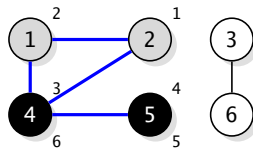
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:     DFS-VISIT( $G, v$ )
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(4)

$time := 6$

Stack chiamate DFS-V: $\langle 4, 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE$ ;  $u.\pi := NULL$   
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

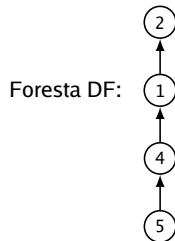
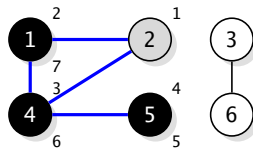
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
7:  $time := time + 1$ ;  $u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(1)

$time := 7$
Stack chiamate DFS-V: $\langle 1, 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE; u.\pi := NULL$   
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

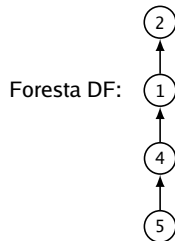
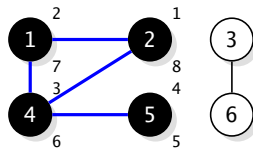
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1; u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
7:  $time := time + 1; u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(2)

$time := 8$
Stack chiamate DFS-V: $\langle 2 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do  
2:    $u.color := WHITE; u.\pi := NULL$   
3:  $time := 0$  // Variabile globale  
4: foreach  $u \in V$  do  
5:   if  $u.color == WHITE$  then  
6:      $DFS-VISIT(G, u)$ 
```

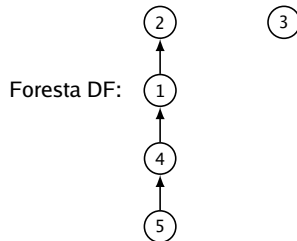
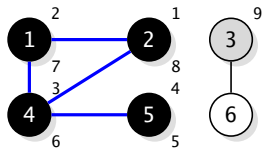
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1; u.d := time$   
2:  $u.color := GRAY$   
3: foreach  $v \in G.Adj[u]$  do  
4:   if  $v.color == WHITE$  then  
5:      $v.\pi := u$   
6:      $DFS-VISIT(G, v)$   
7:  $time := time + 1; u.f := time$   
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(3)

$time := 9$
Stack chiamate DFS-V: $\langle 3 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:     DFS-VISIT( $G, u$ )
```

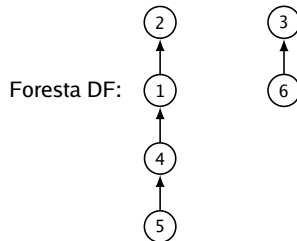
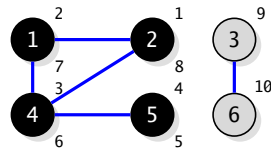
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:     DFS-VISIT( $G, v$ )
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 1-2 di DFS-V(6)

$time := 10$
Stack chiamate DFS-V: $\langle 6, 3 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:      $DFS-VISIT(G, u)$ 
```

Algoritmo: DFS-VISIT(G, u)

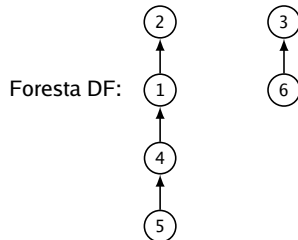
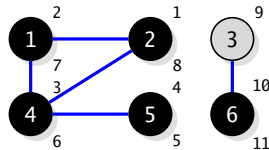
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:      $DFS-VISIT(G, v)$ 
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(6)

$time := 11$

Stack chiamate DFS-V: $\langle 6, 3 \rangle$



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:     DFS-VISIT( $G, u$ )
```

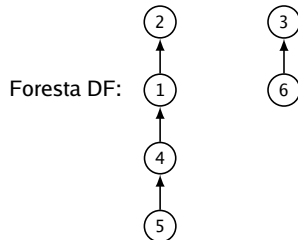
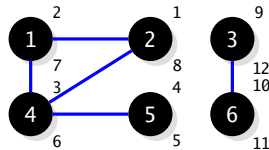
Algoritmo: DFS-VISIT(G, u)

Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:     DFS-VISIT( $G, v$ )
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Dopo esec. 7-8 di DFS-V(3)

$time := 12$
Stack chiamate DFS-V: {3}



Grafi

Visita in profondità: DFS(G)

Algoritmo: DFS(G)

Input: $G = (V, E)$

```
1: foreach  $u \in V$  do
2:    $u.color := WHITE$ ;  $u.\pi := NULL$ 
3:  $time := 0$  // Variabile globale
4: foreach  $u \in V$  do
5:   if  $u.color == WHITE$  then
6:     DFS-VISIT( $G, u$ )
```

Algoritmo: DFS-VISIT(G, u)

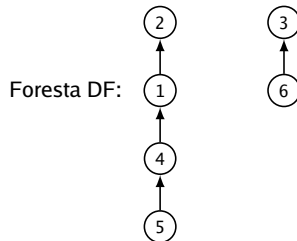
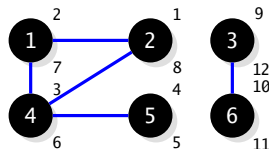
Input: $G = (V, E), u \in V$

```
1:  $time := time + 1$ ;  $u.d := time$ 
2:  $u.color := GRAY$ 
3: foreach  $v \in G.Adj[u]$  do
4:   if  $v.color == WHITE$  then
5:      $v.\pi := u$ 
6:     DFS-VISIT( $G, v$ )
7:  $time := time + 1$ ;  $u.f := time$ 
8:  $u.color := BLACK$ 
```

Fine esecuzione

$time := 12$

Stack chiamate DFS-V: $\langle \rangle$



Grafi

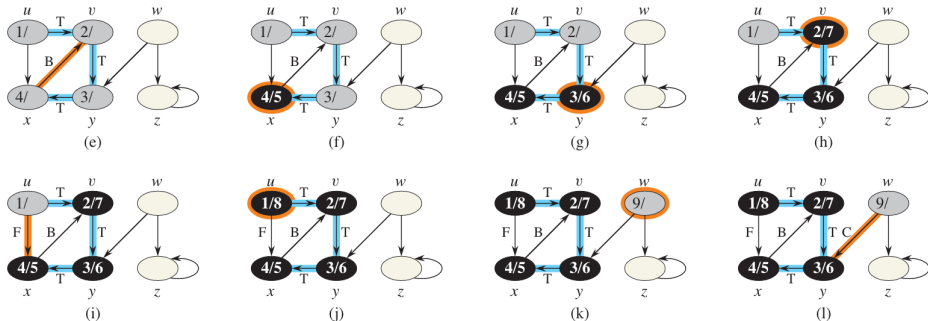
Visita in profondità: estensione di DFS

La figura 20.4 di CLRS mostra un'esecuzione dove gli archi vengono anche etichettati (non dall'algoritmo).

Un arco (u, v) è:

- **T** (tree) quando viene ispezionato e v è bianco.
- **B** (backward) quando viene ispezionato e v è grigio. ($v.d \leq u.d$, v è un antenato di u).
- **F** (forward) quando viene ispezionato, v è nero e $u.d < v.d$ (v è un discendente di u).
- **C** (cross) quando viene ispezionato, il nodo v è nero e $u.d > v.f$ (u e v sono nello stesso albero ma non sono in relazione di discendenza o sono su due alberi distinti).

Parte della figura 20.4 (colore arancione indica l'arco ispezionato o cambio valori di un nodo):



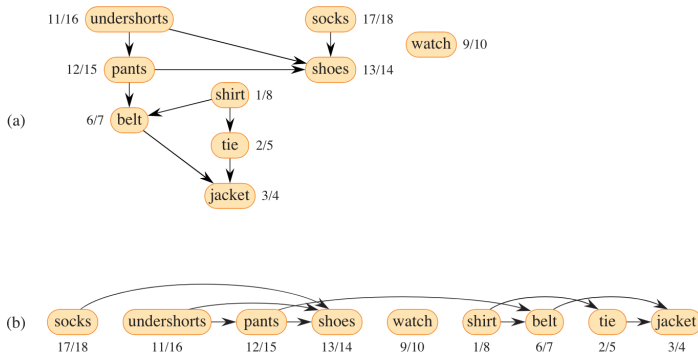
- I risultati di DFS dipendono dall'ordine in cui i vertici sono ispezionati, ma questo non crea problemi (in generale) nelle applicazioni dell'algoritmo.
- L'algoritmo DFS richiede tempo $\Theta(|V|)$ escluso il tempo delle chiamate a DFS-VISIT.
- L'algoritmo DFS-VISIT è eseguito esattamente una volta per ciascun nodo ($|V|$ chiamate).
- In ogni esecuzione di DFS-VISIT, il ciclo **foreach** viene eseguito $|Adj[v]|$ volte, dove v è il nodo in input.
- Sommando $|Adj[v]|$ per tutti i nodi possibili, il ciclo **foreach** viene eseguito $|E|$ volte.
- Il costo totale quindi è $\Theta(|V| + |E|)$.

- I tempi di scoperta e di completamento hanno una **struttura a parentesi**.
 - Se si rappresenta la scoperta di u con $(u$ e il completamento di u con $u)$, allora un'esecuzione di DFS produce un'espressione ben formata.
 - Esempio: l'esecuzione DFS sul grafo G dell'esempio svolto produce l'espressione $'(2 (1 (4 (5 5) 4) 1) 2)(3 (6 6) 3)'$.

Grafi

Ordinamento Topologico

- Si consideri un grafo diretto aciclico (**DAG**, *directed acyclic graph*)
- Un **ordinamento topologico** di un DAG $G = (V, E)$ è un ordinamento lineare dei nodi tale che se $(u, v) \in E$, allora u compare prima di v nell'ordinamento.
 - Se si dispongono i nodi ordinati in una linea, allora gli archi di G sono tutti orientati da sx verso dx.
- Gli ordinamenti topologici sono usati in molti contesti per indicare le precedenze fra gli eventi.



- Il tempo di completamento dei nodi calcolato da DFS è sufficiente per costruire l'ordinamento topologico.

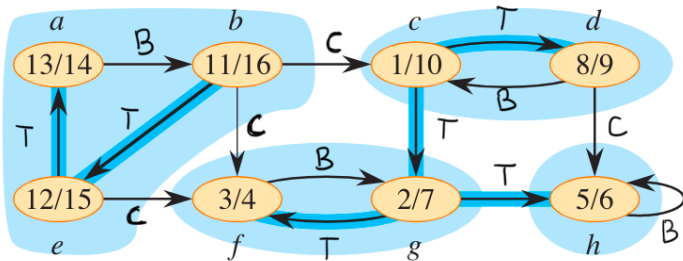
Algoritmo: TOPOLOGICAL-SORT(G)

Input: DAG $G = (V, E)$

```
1: lista :=  $\langle \rangle$   
2: Esegui DFS( $G$ ) e, ogni volta che un  $u.f$  è calcolato,  
3: lista :=  $u + lista$  // Inserisci  $u$  in testa  
4: return lista
```

- La dimostrazione di correttezza si basa sul fatto che, in un DAG, per ogni arco (u, v) vale $u.f > v.f$. Quindi inserire ogni nodo in testa alla lista al momento del completamento produce un ordinamento topologico.
- Poiché il DAG non contiene archi all'indietro, per ogni arco (u, v) la DFS determina $v.f < u.f$ (dettagli nella dimostrazione Teorema 20.12 [CLRS]).

- In un grafo **diretto** $G = (V, E)$, una **componente fortemente connessa** è un insieme massimale di nodi $C \subseteq V$ tali che per ogni coppia $u, v \in C$, si ha che $u \rightsquigarrow v$ e $v \rightsquigarrow u$.



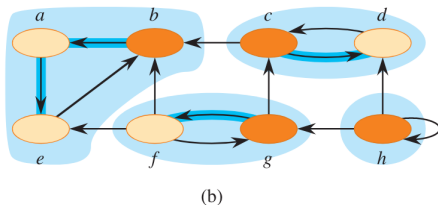
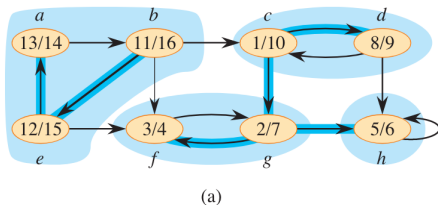
(a)

I nodi della stessa regione azzurra formano una componente fortemente connessa.

Grafi

Applicazione DFS: Componenti fortemente connesse

- STRONGLY-CONNECTED-COMPONENTS determina le componenti fortemente connesse di un grafo diretto $G = (V, E)$ usando il grafo **trasposto** $G^T = (V, E^T)$, dove $E^T = \{(v, u) \mid (u, v) \in E\}^*$.
- G e G^T hanno le stesse componenti fortemente connesse.
- STRONGLY-CONNECTED-COMPONENTS calcola le componenti fortemente connesse eseguendo una DFS su G e una DFS su G^T in tempo $\Theta(|V| + |E|)$.



*Se G è rappresentato con le liste di adiacenza, tempo richiesto per determinare G^T è $\Theta(|V| + |E|)$.

- La correttezza di STRONGLY-CONNECTED-COMPONENTS si basa sui seguenti lemmi.

Lemma 4

Si considerino due componenti fortemente connesse C e C' distinte di $G = (V, E)$. Se $(u, v) \in E$, dove $u \in C'$ e $v \in C$, allora $f(C') > f(C)$, dove $f(U)$ è l'ultimo tempo di completamento di un vertice qualsiasi della componente U .

Dimostrazione.

Se $d(C') < d(C)$ ($d(U)$ è il primo tempo di scoperta di un vertice qualsiasi della componente U), sia nodo $y \in C'$ t.c. $y.d = d(C')$. Tutti i nodi di C' e C sono suoi discendenti. Quindi $y.f = f(C') > f(C)$.

Se $d(C') > d(C)$, sia $y \in C$ t.c. $y.d = d(C)$. Solo i nodi in C saranno suoi discendenti. Vale $y.f = f(C)$. Al tempo $y.d$, tutti i nodi di C' sono bianchi e non può esistere un cammino da y a un nodo in C' (altrimenti non sarebbero due componenti distinte). Al tempo $y.f$ tutti i nodi in C' sono ancora bianchi: verranno scoperti e completati solo dopo $y.f = f(C)$, quindi $f(C') > f(C)$. □

Corollario 2

Si considerino due componenti fortemente connesse C e C' distinte di $G = (V, E)$. Se $f(C') < f(C)$, allora E^T non contiene nessun arco (v, u) tale che $v \in C$ e $u \in C'$.

Dimostrazione.

La contronominale del Lemma 4 dice che se $f(C') < f(C)$ non esiste alcun $(u, v) \in E$ tale che $u \in C'$ e $v \in C$. Le componenti sono le stesse, quindi $(u, v) \in E$ nel trasposto diventa $(v, u) \in E^T$. □

Grafi

Applicazione DFS: Componenti fortemente connesse

- STRONGLY-CONNECTED-COMPONENTS visita DFS G per determinare i tempi di completamento dei nodi.
- Poi visita DFS G^T scegliendo i nodi in ordine decrescente rispetto al tempo di completamento determinato in G .
- Inizia dal nodo x con $x.f$ massimo. Sia C la componente che contiene x .
- $f(C)$ è massimo. La visita in profondità da x visita solo i nodi di C per il Corollario 2. Quindi l'albero DFS con radice x determina una componente.
- Nel prossimo **for** principale di DFS(G^T), si sceglie un nodo y tra i rimanenti con $y.f$ massima.
- La visita in profondità da y non potrà andare in componenti con nodi bianchi; potrà al più incontrare archi verso nodi già neri di C . Ancora una volta, considerando l'albero DFS con radice y , si ottiene un'altra componente.

Teorema 2

STRONGLY-CONNECTED-COMPONENTS *calcola correttamente le componenti fortemente connesse di un grafo orientato G .*

Algoritmo: STRONGLY-CONNECTED-COMPONENTS(G)

Input: $G = (V, E)$

Output: Componenti fortemente connesse di G

```
1: DFS( $G$ )                                     // Determina i tempi di completamento  $u.f$ 
2: Determina  $G^T$ 
3: DFS( $G^T$ ) esaminando i nodi in ordine decrescente rispetto al tempo di completamento
4:  $C := []$ 
5:  $i := 0$ 
6: foreach albero DFS  $T$  in  $G^T$  do
7:    $C[i++] :=$  sottografo indotto dai nodi dell'albero  $T$ 
8: return  $C$ 
```

La complessità in tempo è data dalle due visite DFS: $\Theta(|V| + |E|)$.

Infatti ciascun $C[i]$ può essere determinato durante la seconda visita DFS. Nell'algoritmo, C viene determinato con un ciclo **foreach** solo per chiarezza.

- Studiare le sezioni 20.1, 20.2, 20.3, 20.4 e 20.5 [CLRS] tranne le dimostrazioni dei teoremi 20.7, 20.9 e 20.10.
- Risolvere almeno gli esercizi 20.1-1, 20.1-3, 20.1-5, 20.2-3, 20.2-5.
- Scrivere la procedura che stampa il cammino minimo dalla sorgente a un nodo dato (se esiste).